

Blockchain Document

Acknowledge to



<https://bitcoin.org/bitcoin.pdf>

<https://bitcoin.org/bitcoin.pdf>

[Ethereum White Paper - A Next-Generation Blockchain Service Contract and Decentralized Application Platform, by Vitalik Buterin, et al](#)



<https://github.com/ethereum/go-ethereum>

<https://github.com/ethereum/go-ethereum>



[Bitcoin](#)

[Bitcoin](#)



<https://github.com/bitcoin/bitcoin>

<https://github.com/bitcoin/bitcoin>

1.What's Blockchain?

1.1 Definition & Core feature

区块链本质上是一个**分布式的账本数据库**，网络中的每个节点都拥有一本完整的账本。[账本是用来记录交易的!]

- **一致性 [Consistency]**: 全球节点通过共识机制保持账本数据的一致。
- **不可篡改性 [Immutability]**: 一旦数据被记录在区块链上，就无法轻易修改和删除。
- **分布式/去中心化 [Decentralization]**: 去中心化的拓扑结构，没有任何单一节点可以控制整个网络。
- **匿名性 (Anonymity)**: 交易双方均不需要公开真实身份就可以进行交易。

AI 是一种生产力，没有改变现有的组织结构，Blockchain 是一种生产关系，挑战并改变了人类现有的组织模式和信任机制。

1.2 Classification

- **Public Chain[公有链]**: 任何人均可作为节点加入, 参与数据验证、共识和读写 (如比特币、以太坊)。
- **Consortium Chain[联盟链]**: 由几个特定的机构作为节点共同参与共识和记账
- **Private Chain[私有链]**: 链上数据读写权限由单一组织控制, 去中心化程度最低, 适用于企业内部的审计和数据管理。

深圳区块链电子发票 / 杭州互联网法院将证据信息进行分布式存储以防篡改。

1.3 Preliminary of Bitcoin

受 Adam Back 的 Hashcash 和 Wei Dai 的 B-Money 启发, 中本聪 [Satoshi Nakamoto] 于 2008 年提出比特币, 2009 年 1 月创世区块诞生。其与以往数字货币的最大区别在于: **完全 P2P**, 彻底摆脱对第三方信任机构的依赖, 并完美解决了双花问题 [Double-Spending]。

Bitcoin 主要依赖两种密码学技术: Hash Function & Digital Signature

Digital Signature

- **非对称加密**: 包含公钥 (pk) 和私钥 (sk)。适合加密少量数据和身份验证。
- **比特币钱包验证**: 发送者使用自己的私钥 (sk) 对交易信息进行签名 (Sign), 生成无法伪造的数字串。接收者和其他节点通过发送方的公钥 (pk) 来验证该签名的合法性。

💡 比特币钱包背后的签名和验证机制, 是整个 Blockchain 信任的基础。主要依赖于椭圆曲线数字签名算法 ECDSA-Elliptic Curve Digital Signature Algorithm, 特别是比特币使用的一条特定曲线 secp256k1。

sk, G [椭圆曲线上一个固定基点, 比特币中公开], pk ($pk = sk * G$, 注意, 在比特币中, 使用 pk 反推 sk 是几乎不可能的)

为什么是不可能的? 因为使用了一种 Trapdoor Function, 这里的乘法不是算术乘法, 而是椭圆曲线上的几何跳跃。

- **地址生成**: 比特币用户的身份 (地址) 由公钥经过哈希运算生成: $Address = Hash(pk \parallel x)$ 。

Hash Function

将任意长度的消息映射为较短的定长数据 (如 256 bit 的 SHA-256), 相当于生成数据的 “数字指纹”。

- **碰撞阻力 (Collision Resistance)**: 在计算能力范围内, 极难找到两个不同的输入得到相同的哈希值。即: 若 $x \neq y$, 则极大可能 $H(x) \neq H(y)$ 。

- **隐秘性 (Hiding)**: 给定 $H(x)$ 无法反推 x 。为了防止输入空间过小被穷举，通常引入具备高阶最小熵（高度随机）的 `nonce`。
- **谜题友好性 (Puzzle-Friendliness)**: 无法通过规律预测哈希值，只能暴力穷举。这是比特币工作量证明 (PoW) 的核心基础。

延伸：承诺协议 (Commitment Protocol)

类比: 信息 (msg) 是金条，密文 (com) 是保险柜，随机数 (nonce) 是唯一的钥匙。

承诺阶段: 计算并公布 $com = Hash(nonce \parallel msg)$ 。利用 **Hiding** 特性，别人看到 com 猜不出 msg。

验证阶段: 公布 msg 和 nonce[在这种setting下, nonce也需要保留下来]。大家重算 $Hash(nonce \parallel msg)$ 是否等于 com。利用 **Binding**（基于碰撞阻力）特性，发送者无法找到另一套 msg 和 nonce 来伪造之前的承诺。

1.4 Data Structure of Bitcoin

哈希指针 (Hash Pointer)

普通链表的指针仅记录下一个数据的物理地址。而哈希指针不仅包含地址索引，还包含被指数数据的哈希值： $*ptr = H(data)$ 。

这使得系统不仅能找到上一个区块，还能**验证上一个区块的数据是否被篡改**。需要注意的是，区块的物理地址在不同节点的设备上是不同的，全节点通过索引查找该哈希值对应的区块位置。

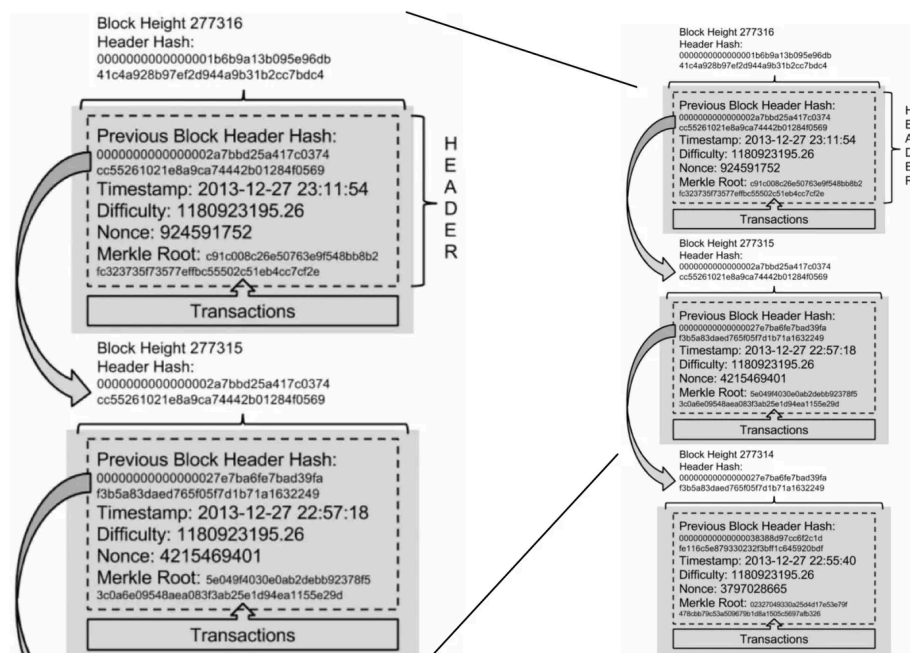


区块结构 (Block Structure)

- **创世区块 (Genesis Block)**: 第一个区块，静态硬编码在客户端软件中，绝对无法修改。
- **区块头 (Block Header) - 包含 6 个重要字段**:
 - a. `Version` (4 字节): 软件 / 协议版本。
 - b. `Previous Block Hash` (32 字节): 指向上一个区块头的哈希指针。[是对上一个区块的 Header 整体做的 Hash]
 - c. `Merkle Root` (32 字节): 区块内所有交易构成的默克尔树的根哈希值。
 - d. `Timestamp` (4 字节): 区块产生的近似时间。
 - e. `Difficulty Target` (4 字节): PoW 的门槛难度。

f. **Nonce** (4 字节): 矿工为满足难度目标而不断尝试的随机数。

- **区块体 (Block Body)**: 具体的交易 (Transactions) 列表。

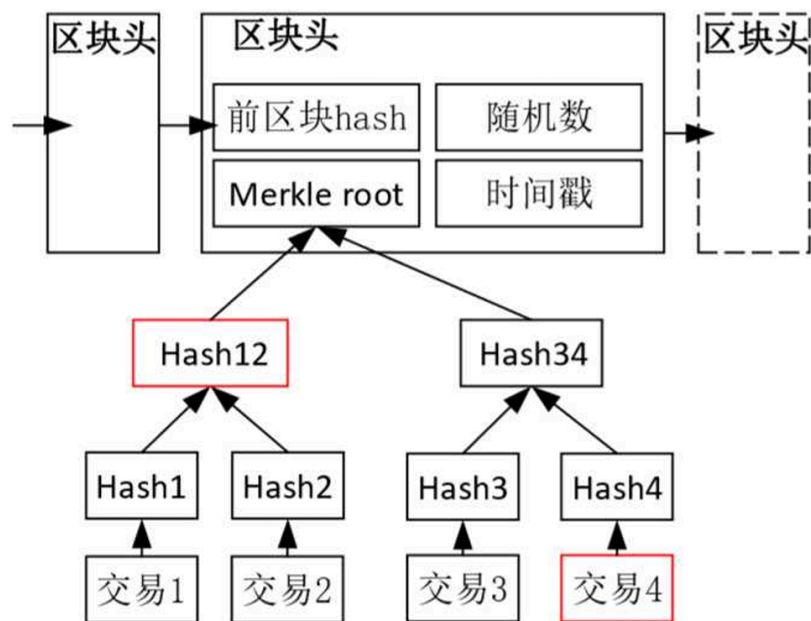


Block Header的结构

默克尔树 (Merkle Tree) 防篡改机制

将区块内的交易两两取哈希，层层递进，最终在顶部形成唯一的 Merkle Root Hash。

- **区块内防篡改**: 修改任意一笔交易，都会导致向上的哈希值雪崩式改变，最终导致 Merkle Root 改变。
- **区块间防篡改**: 一旦 Merkle Root 改变，当前区块头的哈希值也会改变。由于下一个区块头包含了指向当前区块的 **Previous Block Hash**，这将导致后续所有区块的哈希值全部失效。



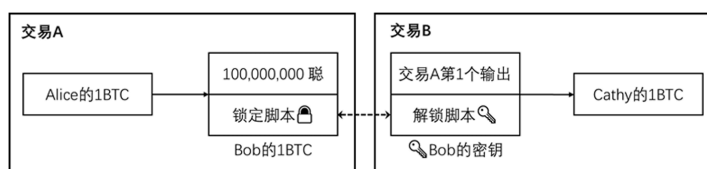
Merkle Tree的防篡改

1.6 交易模型：UTXO 模型

比特币没有账户余额的概念，而是采用了 **UTXO (Unspent Transaction Output, 未花费的交易输出)** 模型。

一笔交易分为两部分：

- **Input (输入 - 钱从哪来)：** 必须引用并解锁之前已完成交易的 Output。包含指向前一个交易的哈希与索引[输入对前一个交易输出的引用]，以及 **Unlocking Script (解锁脚本)** (通常是用户的私钥签名)。
- **Output (输出 - 钱去哪了)：** 包含数额和 **Locking Script (锁定脚本)** (定义了只有拥有对应密钥的接收方才能在未来解锁使用这笔钱)。



输入对前一个交易的输出的引用



多输入交易

共识机制、挖矿与分叉

共识的底层逻辑

完美共识的三要素：**Termination (有限时间结束)**、**Agreement (诚实节点决策一致)**、**Validity (结果必须是合法提案)**。

- **拜占庭容错 (BFT-based)**: 依赖通信和签名 (如 PBFT), 容错率 $3f+1 \leq n$ 。效率高但扩展性差。
- **领导者选举 (Leader Election-based)**: 比特币的选择。通过算力竞争获取记账权, 赢家打包区块。扩展性极强, 但性能较慢。

PoW 挖矿与难度调整

矿工不断尝试改变区块头中的 `Nonce`, 使得:

$\text{SHA}(\text{Merkle Root} + \text{PrevHash} + \text{Timestamp} + \text{Nonce}) < \text{Target}$ [Target 越小, 前导 0 越多, 难度越大]

- **无记忆性与 Mempool**: 每次哈希碰撞是独立的。当新区块被全网广播后, 节点会从自己的交易池 (Mempool) 中剔除已上链的交易。
- **难度调整**: 每 2016 个区块 (约两周) 系统自动调整一次 Target, 确保出块时间稳定在 10 分钟左右。

攻击防御机制

- **女巫攻击 (Sybil Attack)**: 恶意者伪造大量节点投票。**防御**: 放弃按人头投票, 改为按算力投票 (PoW)。
- **双花攻击 (Double-Spending)**:
 - **同等验证 + 最长链原则**: 如果 Gloria 发起双花, 诚实节点收到冲突交易会丢弃后者。几万个矿工中大概率是诚实矿工先解出题并广播合法区块。即便 Gloria 用恶意算力强行挖出一个包含双花交易的区块, 只要诚实节点拒收, 该区块就会成为“孤块 (Orphan Block)”, 其消耗的电力成本打水漂。
 - **6 区块确认**: 即使恶意者试图在链后秘密分支并疯狂挖矿以期超越主链, 随着后面累积的区块越来越多 (达到 6 个), 其算力追上的概率呈指数级下降。

经济模型与博弈论

- **通缩模型**: 初始区块奖励 50 BTC, 每产生 21 万个区块 (约 4 年) 奖励减半, 总量恒定 2100 万。矿工收益 = 出块奖励 + 交易手续费。
- **博弈约束**: 获取记账权需要付出巨大的电力和硬件沉默成本, 这迫使掌握算力的人更倾向于诚实守信以获取系统奖励, 作恶一定是亏本买卖 (哪怕发起 51% 攻击也会导致比特币信任崩塌、币价暴跌, 作恶者手中的币也会变得一文不值)。
- **CAP 与 FLP 原理的取舍**: 在分布式异步系统中, 比特币牺牲了绝对的安全性 / 一致性 (存在极小概率的分叉风险, 需要等待 6 个区块达到最终一致), 换取了高可用性 / 活性 (网络始终能稳定出块)。

节点类型与分叉

- **节点类型：**
 - **全节点 (Full Node)：** 保存创世区块以来所有数据，独立验证每一笔交易，通常由志愿者无偿运行。
 - **轻节点 (SPV Node)：** 只下载区块头，利用默克尔树验证交易是否被打包，不维护全网历史。
 - **挖矿节点 (Mining Node)：** 唯一职责是进行哈希碰撞拼算力，无义务维护历史记录。
- **分叉 (Forking)：**
 - **软分叉 (Soft Fork)：** 向前兼容的“打补丁”。规则变严格，老节点由于利益驱动通常会避免做无用功而被迫升级，临时分叉最终会消失并合并入最长链。
 - **硬分叉 (Hard Fork)：** 新旧节点互不相容（如调整区块大小上限），导致区块链永久性物理分裂，变成两种不同的加密货币（如 BTC 与 BCH）。

2.What's Ethereum and Smart Contracts

比特币的设计极度追求安全，脚本语言缺乏图灵完备性，无法执行复杂逻辑。以太坊作为“区块链 2.0”，最大的突破是引入了**智能合约 (Smart Contracts)**，成为了一个“交易驱动的状态机”。

2.1 智能合约 (Smart Contracts)

由 Nick Szabo 提出，是一段存放在区块链上的代码，依托区块链系统在参与者之间实现对执行的一致认可。

尼克·萨博的“自动售货机”类比：

投入硬币并按下按钮（事件驱动） $\rightarrow$ 机器验证并掉落饮料（价值转移与自动执行）。

智能合约（常用 Solidity 编写）将这种机制部署到区块链上。只要触发条件，代码就必然忠实执行（Code is law），彻底解决了人与人之间的信任问题。

2.2 账户模型 (Account Model)

不同于比特币的 UTXO，以太坊采用了更接近银行的**账户模型**。

- **外部账户 (EOA - External Owned Account)：**
 - 由普通用户的私钥控制。
 - 包含：**Balance** (余额) 和 **Nonce** (累积发起的交易次数，用于防止重放攻击)。
 - **主动性：** 只有 EOA 能主动触发交易（转账或调用合约）。

- **合约账户 (Contract Account):**

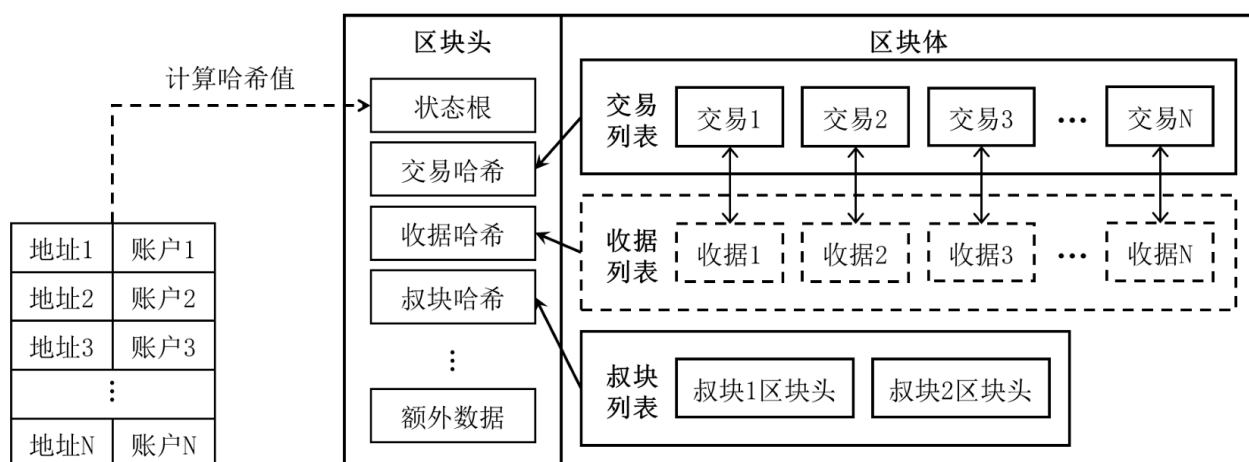
- 没有私钥，由部署在链上的智能合约代码控制。
- 除了 Balance 和 Nonce，还包含：CodeHash (代码的哈希值) 和 Storage Root (维护智能合约状态的存储树根节点哈希)。
- **被动性**：不能主动发起交易，只能在被 EOA 或其他合约调用时触发执行。

2.3 数据结构：MPT 树与三种收据

以太坊全节点需要维护所有账户的状态。如果每次状态改变都重构整棵 Merkle Tree 代价极大。因此以太坊采用了 **MPT (Merkle Patricia Trie, 默克尔压缩前缀树)**，结合了 Trie 的高效查找 / 局部更新特性和 Merkle Tree 的防篡改特性。

在每个区块的区块头中，包含三棵关键 MPT 树的 Root Hash：

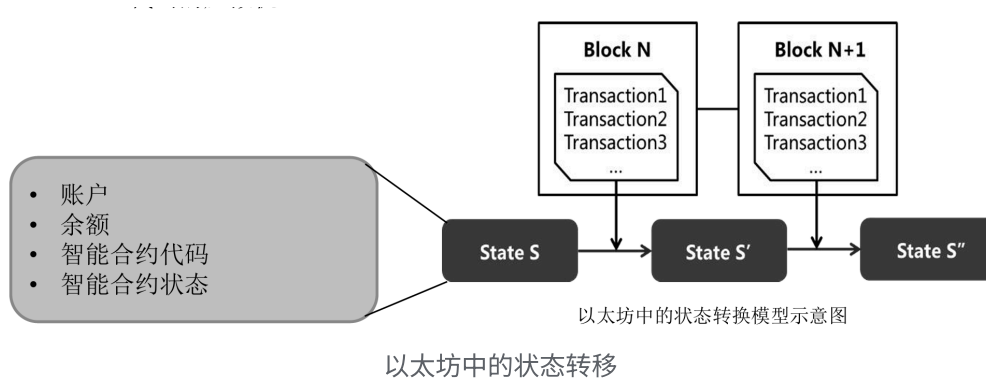
1. **状态树 (State Tree)**：记录所有账户当前状态。新区块产生时只局部更新改变的分支，生成新的状态树，大部分未改变的节点与旧树共享。极大地便利了分叉时的状态回滚。
2. **交易树 (Transaction Tree)**：记录当前区块打包的所有交易。
3. **收据树 (Receipt Tree)**：记录交易执行后的中间状态、结果及合约生成的日志。
 - **布隆过滤器 (Bloom Filter)**：以太坊在区块头和收据中使用了布隆过滤器（特性：可能存在或绝对不存在），使得轻节点无需下载完整数据，就能极快地检索特定日志或事件。



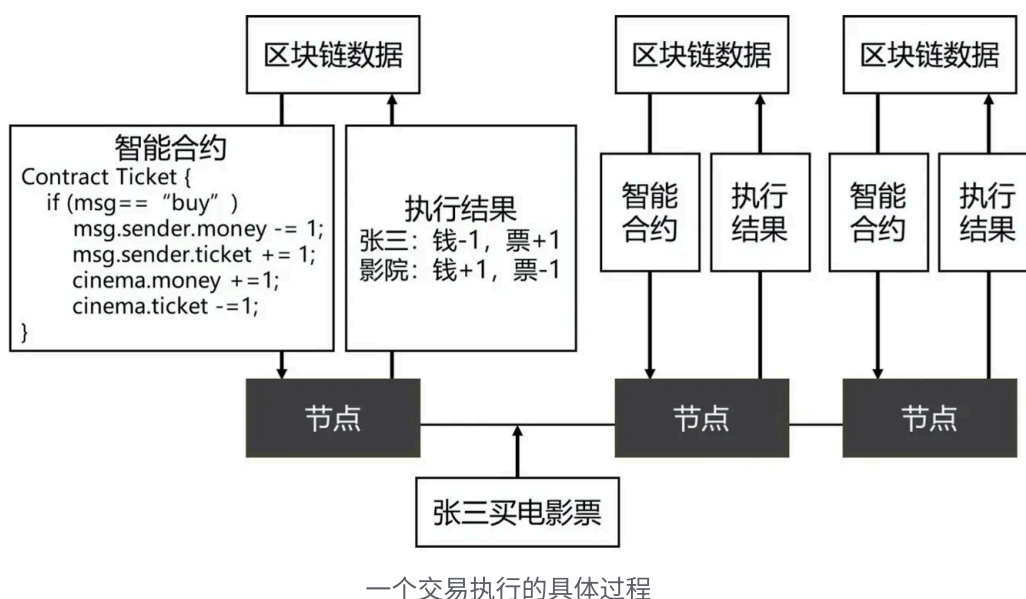
以太坊的区块结构

2.4 交易与以太坊虚拟机 (EVM)

以太坊是一个**交易驱动的状态机**：系统从状态 SSS ，经过 EVM 执行区块中的交易集合，确定性地转换为下一个状态 $SS'S$ 。



智能合约就是驱动状态机转换的代码。



每个节点独立维护区块链数据，“张三买电影票”的交易在所有节点中被独立验证。以太坊虚拟机的执行结果将以某种方式写回到区块链数据中，比如张三的余额、影院的余票等。每个区块中会保留一段摘要，这段摘要为执行完区块中交易后以太坊状态的 stateRoot，任一子状态的不同都将导致 stateRoot 的不同。

交易 (Transaction) 的核心字段

- `from` / `to` / `value` / `v, r, s` (发送者签名)
- `data`：附带数据，用于传递创建合约的机器码，或调用合约的具体函数及参数。
- `gasLimit` 与 `gasPrice`：为了防止智能合约陷入死循环（停机问题），每笔交易必须预设愿意消耗的最大 Gas 量和单价。执行完毕后，剩余 Gas 退回；若 Gas 耗尽，状态回滚但手续费被矿工扣除。

交易的三种类型

1. 普通转账交易：`to` 指向另一个外部账户 (EOA)。

2. **创建合约交易：** `to` 字段为**空 (null)**。以太坊网络识别后，EVM 会自动生成一个新的合约地址，并将 `data` 中的代码永久存储进去。至此，State Tree 上诞生了一个全新的合约账户。
3. **调用合约交易：** `to` 指向一个合约账户。EVM 会根据 `data` 字段提取对应的函数签名和参数，执行该智能合约的代码逻辑。